



Anil Seth

A Software Engineer's Tool-kit

Software engineering recommendations have not changed. The issues and concerns have been known for years. However, with the availability of excellent tools, it is now no longer a pain to implement and follow the recommended practices, starting with requirements analysis.

The requirements analysis phase is perhaps the most important activity, due to its impact on the success of a project. It is also the most amorphous. Software engineers should be able to organise their thoughts and analysis in a manner that is clear to the team. They should be able to communicate their understanding to the customer or user, in order to be able to get appropriate feedback.

Freemind (<http://freemind.sourceforge.net>) or similar memory map tools can be invaluable for both these needs. The tree-structured presentation is very concise, making it easy for people to comprehend the whole picture. Also, it allows one to easily expand each branch to view more details selectively. The use of tags and notes allows the mind to focus, and not be overwhelmed by excess data. The one issue that may be a challenge is to get an organisation to add memory maps as an acceptable way of communicating requirements.

Requirements need to be converted into an estimate of effort and time. There is also a need for a plan. A useful tool for these cases is *Planner* (<http://live.gnome.org/Planner>), which is included in most distributions. I personally liked *Faces* (<http://faces.homeip.net>) very much. It is still worth exploring, even though it has been dormant for the last few years. It appealed to me as the plan is in Python code, and as a consequence, making changes in the plan was incredibly simple! While it may be hard to get an organisation to use *Faces*, by using it, a programmer may find even planning, fun.

Execution

The second part of project execution is keeping track of the project's status. For this, *Trac* (<http://trac.edgewall.org>) is a wonderful tool; it combines a wiki with a ticketing system. The latter is invaluable for tracking tasks, issues and change requests. *Trac* includes the ability to browse the source from a Subversion repository. Needless to say, a plug-in allows for browsing a Git repository as well.

Subversion (<http://subversion.tigris.org>) and *Git* (<http://git-scm.com>) are great options for keeping the source in a repository and ensuring that all changes are tracked. It is rare to find programmers who have not needed to undo the changes they have made, and who haven't struggled to trace the changes. There is really no excuse for not using a source code repository these days.

The excellent integration of unit testing tools with IDEs makes it very easy to code unit tests as a part of programming. All software engineers should be familiar with Java *JUnit* (<http://en.wikipedia.org/wiki/Junit>) or a unit-testing framework for their

preferred language. Martin Fowler's comment about JUnit says it all: "Never in the field of software development have so many owed so much to so few lines of code."

Verification

It is really irritating these days to come across Web applications that will only work correctly in IE. It is even worse, and rather foolish, for a major organisation to insist on the use of only IE. Users today are no longer on the desktop or on notebooks. It would not surprise me if, even in India, a larger numbers of users are on smart phones!

Even if one only supports IE on a desktop, verifying a website manually just does not make sense, given that there are superb tools like *Selenium* (<http://seleniumhq.org>). Once these tools are in use, there will be no lethargy in running regression tests.

Verifying one's own work is extremely hard—not because we think we are perfect, but rather, we do not notice defects we never intended. We frequently read what we believe we have written, and not what is actually written! On the other hand, verifying someone else's work is tedious, and may even seem unpleasant. It is hard to keep emotional reactions and personal equations separate from the task of finding defects in code, especially in a face-to-face review. However, a couple of applications have made the task of peer reviews much easier. These are *ReviewBoard* (<http://www.reviewboard.org/>) and *Gerrit* (<http://code.google.com/p/gerrit>), which is actually a fork/rewrite of *Rietveld* (<http://code.google.com/p/rietveld>).

The immense benefit of these tools is the integration with source code repositories. Once a code file is modified, one does not need to review the whole file again, but only focus on the changes.

In my biased opinion, the list of tools featured in this article comprises the minimum that those writing software should know, unless they graduated in the previous century. I would prefer it if no one is given a computer engineering or IT degree unless they demonstrate proficiency in this set of tools, or their equivalents.

In the long run, those who do not use such tools will be far too inefficient to compete with those who do. 

By: Anil Seth

The author currently is a visiting faculty at IIT-Ropar, prior to which he was a professor at Padre Conceicao College of Engineering (PCCE) in Goa. He has managed IT and imaging solutions for Phil Corporation (Goa), and worked for Tata Burroughs/TIL. You can find him online at <http://sethanil.com> and reach him via email at anil@sethanil.com.